Prompt: "Generate a detailed system architecture diagram for an AI-powered study assistant web app with retrieval augmented generation (RAG). The system allows users to sign in, upload study documents like PDFs, PowerPoint slides, Word docs, and handwritten notes. Uploaded documents are processed through a pipeline that includes text extraction, OCR for handwriting, and smart chunking that preserves document structure like page numbers and slide titles. Chunks are embedded and stored in a vector database (such as FAISS or Qdrant). A keyword search index (using BM25) complements retrieval. The system implements hybrid retrieval combining semantic and keyword search with cross-encoder reranking. User queries are sent via the web frontend to a backend API server (FastAPI), where a caching layer (Redis) checks for cached responses by exact or semantic match. If cache misses occur, queries undergo the hybrid retrieval pipeline, with a fallback web search for low-confidence queries using academic sources (SerpAPI or Tavily). The retrieved context and query are sent to a large language model (LLM) for response generation with prompt engineering that enforces citations. The system stores user sessions, chat histories, and usage analytics in a relational database (PostgreSQL or SQLite). The app supports rate limiting, user authentication with JWT, and detailed monitoring with latency and cost metrics tracked. Visually display components: User browser with frontend UI (chat window, upload button) Backend API server Document processing pipeline (OCR, chunking) Vector DB + Keyword index Cache layer (Redis) LLM service External web search API Session and user DB Show data flow arrows connecting these components from user input to final answer rendering." Please generate Mermaid diagram code for this architecture based on the detailed prompt above